

08. APIs – II

Ciencia de datos II

**Jorge Juvenal
Campos Ferreira**

✉ juvenal.campos@tec.mx

Librerías a descargar

- * ***inegiR*** - API de INEGI
- * ***tidyquant*** - datos financieros
- * ***lubridate*** - fechas
- * ***httr2*** - funciones adicionales a httr
- * ***ellmer*** - chat con LLMs
- * ***plotly*** - gráficas interactivas
- * ***sf*** - mapas

Ejercicio práctico 2:

- * Utilice el API del INEGI para determinar que entidad de la república tiene la mayor aportación al PIB nacional.
- * La librería tidyquant nos permite acceder a varias API de datos financieros (Yahoo finance, FRED). Genere consultas que le permitan ver el precio de la acción de Apple para el último año.
- * Utilice el API de twitterapi.io para poder extraer tweets de algún tema de interés (por ejemplo, los últimos tweets de Elon Musk).
- * Utilice el API de Claude para poder chatear con la IA desde la consola de R. ¿Que aplicaciones considera que podrían hacerse con este acceso?

Ejercicio práctico 2:

Tarea (no tarea):

- * Consiga sus credenciales de la API de Youtube.
- * Con la API de Youtube y el paquete *{tuber}*, descargue los comentarios de algun video relacionado con la polémica de la situación actual en Sinaloa
- * Con la API de Gemini y el paquete *{ellmer}*, determine si los comentarios son ofensivos.
- * Con Ollama, genere una consulta interna para realizar el mismo proceso.

Conceptos a tomar en cuenta:

Loops o bucles en R con *for*

Suponga que quiere hacer un *loop* en R

Paso 1. Definir qué quiero hacer (y qué resultado esperar).

(Generalmente hacemos loops para automatizar tareas repetitivas)

Paso 2. Hago ese proceso para un caso $n = 1$.

(Programo una vez ese proceso para un caso particular)

Paso 3. Identifico las partes variables del proceso.

(¿Qué cambiaría en el caso $n=2, 3, 4, \dots N$)

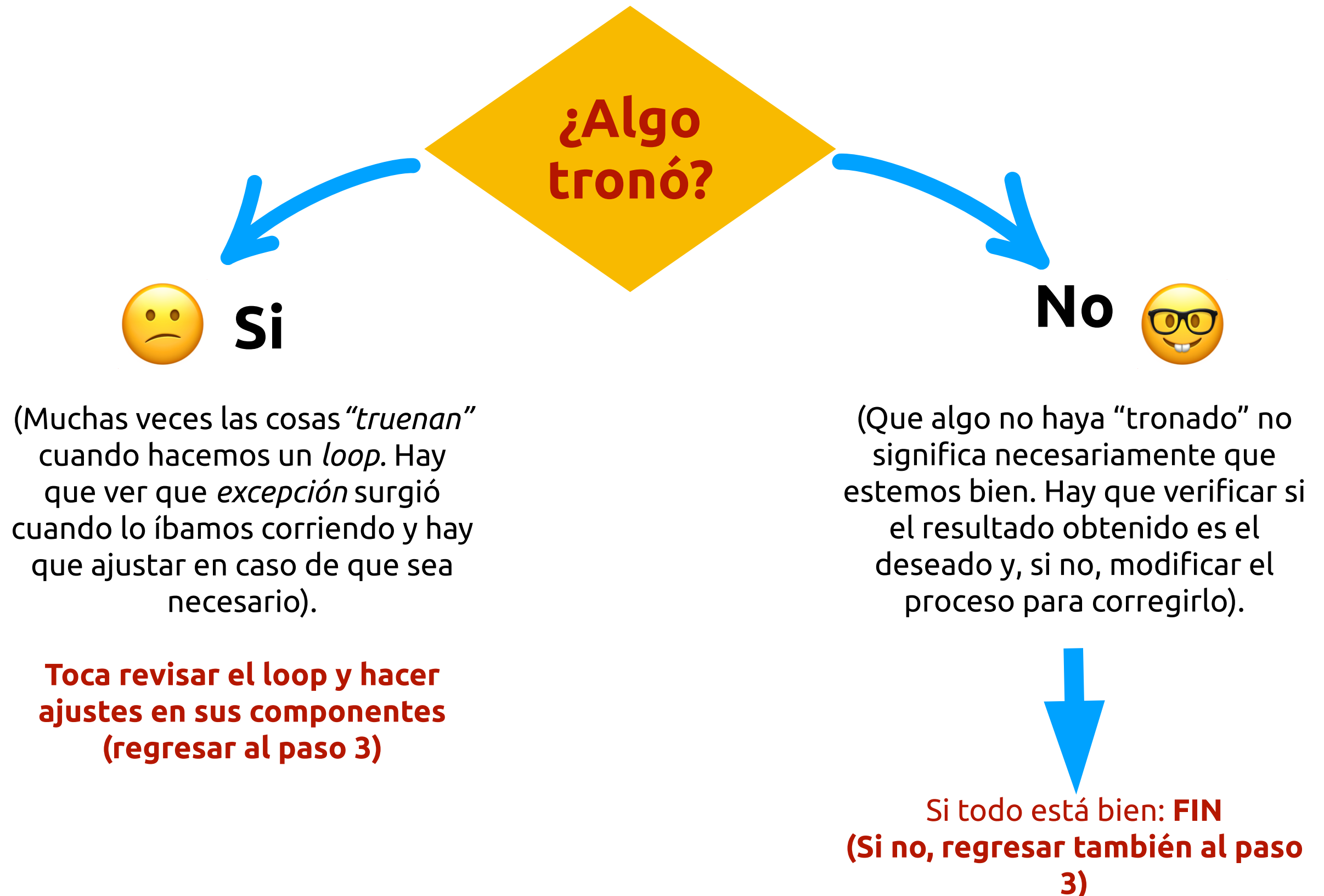
Paso 4. Cambio las partes variables del paso 3 por objetos de R.

(Que irán cambiando en el loop)

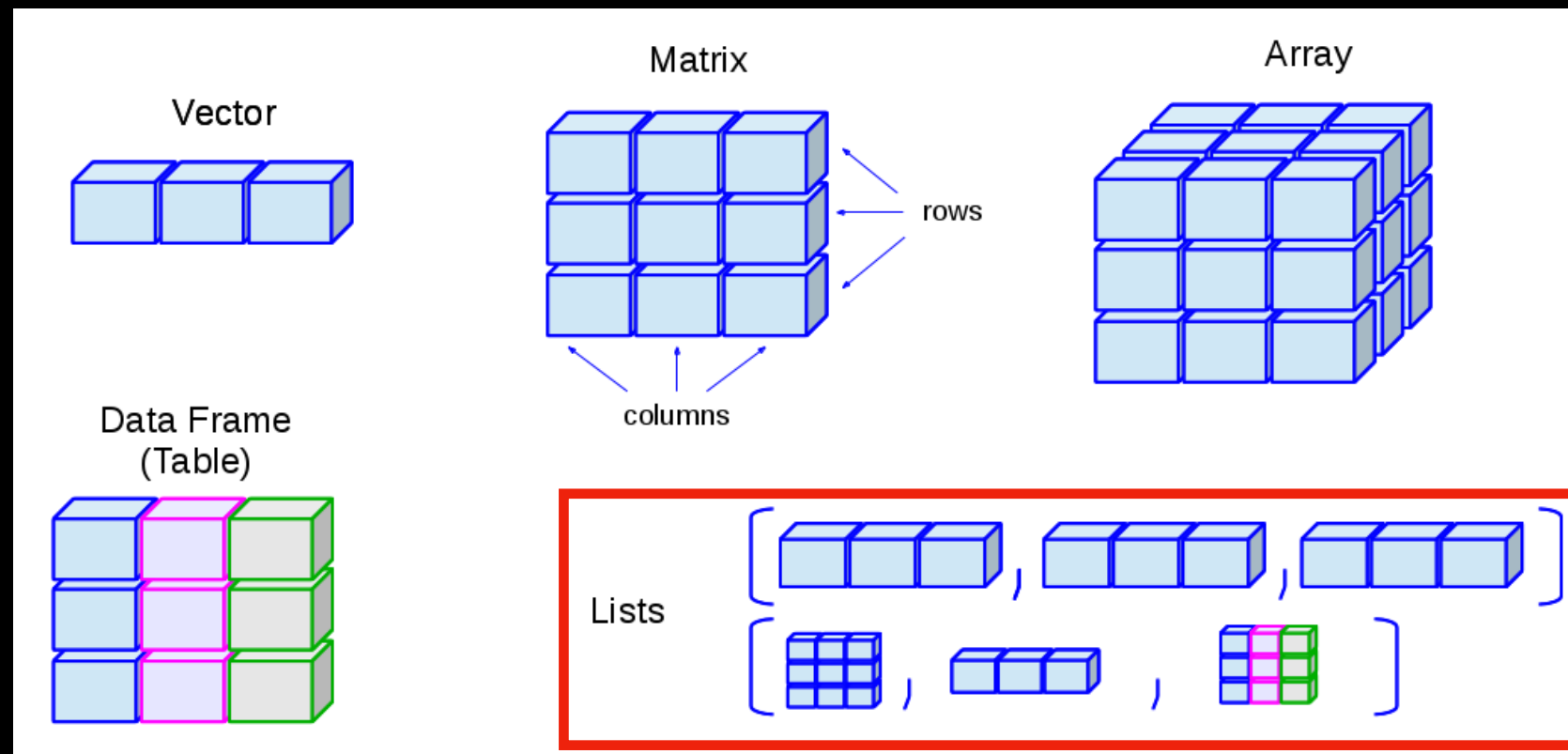
Paso 5. Construyo el *for*

(O uso alguna otra función de *loop*)
definiendo secuencia, contador y proceso

Loops o bucles en R con *for*



Listas



Una lista es como un vector, en donde el contenido de cada casilla puede ser de **cualquier** tipo (otra lista adentro, un vector, una tibble o dataframe, etc.).

Formato JSON



```
{
  "users": [
    {
      "userId": 1,
      "firstName": "Chris",
      "lastName": "Lee",
      "phoneNumber": "555-555-5555",
      "emailAddress": "clee@fileinfo.com"
    },
    {
      "userId": 2,
      "firstName": "Action",
      "lastName": "Jackson",
      "phoneNumber": "555-555-5556",
      "emailAddress": "ajackson@fileinfo.com"
    }
  ]
}
```

```
"store": {
  "book": [
    {
      "category": "reference",
      "author": "Nigel Rees",
      "title": "Sayings of the Century",
      "price": 8.95
    },
    {
      "category": "fiction",
      "author": "Evelyn Waugh",
      "title": "Sword of Honour",
      "price": 12.99
    },
    {
      "category": "fiction",
      "author": "Herman Melville",
      "title": "Moby Dick",
      "isbn": "0-553-21311-3",
      "price": 8.99
    }
  ]
}
```

Ejemplos:

https://github.com/JuveCampos/Shapes_Resiliencia_CDMX_CIDE/raw/master/Infraestructura/Pozos_Sacmex2.geojson;
Ver archivo carpeta local.

Formato JSON



Un archivo *.json (JavaScript Object Notation) es un **archivo que almacena estructuras simples de datos**.

- Este formato es un **estándar para el intercambio de información**, y se usa para transmitir información entre una aplicación y un servidor.
- Estos archivos son **ligeros, basados en texto, legibles por humanos** y pueden ser editados por un editor de texto.
- Es el **formato** que se utiliza para **dar información proveniente de un API**.
- Para leerlos **en R**, tenemos que usar funciones de la librería **{jsonlite}**.

```
{
  "tag": "select",
  "name": "ctl00$plnMain$ddlClasses",
  "id": "plnMain_ddlClasses",
  "class": "sg-combobox",
  "children": [
    {
      "tag": "option",
      "selected": "selected",
      "value": "ALL",
      "html": "(All Classes)"
    },
    {
      "tag": "option",
      "value": "1319202|1",
      "html": "0020A - 36   STUDY HALL (INSTRUCT)"
    },
    {
      "tag": "option",
      "value": "1319203|1",
      "html": "0020B - 33   STUDY HALL (INSTRUCT)"
    },
    {
      "tag": "option",
      "value": "1232138|1",
      "html": "0113A - 1   APENGLAN A"
    },
    {
      "tag": "option",
      "value": "1232137|1",
      "html": "0113B - 1   APENGLAN B"
    },
    {
      "tag": "option",
      "value": "1229605|1",
      "html": "0284A - 2   PRE CALC PREAP A"
    },
    {
      "tag": "option",
      "value": "1229608|1",
      "html": "0284B - 2   PRE CALC PREAP B"
    },
    {
      "tag": "option",
      "value": "1229088|1",

```

Variables de ambiente

★ **Variable de ambiente:** Son pares clave-valor que el sistema operativo o que R utilizan para almacenar información de configuración.

Funcionan como un contexto que afecta a la ejecución de tu sesión en R, sin necesidad de estar definidas en el script.

Variables de ambiente

Una variable normal existe solo dentro de tu entorno de trabajo.

Una variable de ambiente vive en el sistema y es leída por R. Puede afectar como funcionan los paquetes sin que tengas que ponerlas explícitamente en el código.

Suelen usarse típicamente para guardar llaves de acceso a APIs. Tiene la ventaja de la **seguridad** (evitas poner tus contraseñas en el código) y de **mantenibilidad** (cambias solo una contraseña en vez de cambiar la llave en cada código).

Variables de ambiente

Ejemplo de cómo se vería el archivo .Renviron que carga las variables de ambiente.

Para modificar el archivo .Renviron utilizamos la función `usethis::edit_r_environ()`

```
# Ejemplo en un archivo .Renviron o en el sistema
R_LIBS_USER=/Users/juvenal/R/Library
DB_HOST=database.myserver.com
DB_USER=analyst_user
DB_PASS=SuperSecreta123
API_KEY_YOUTUBE=AIZAaSyAxxxxxxx-xxxxx-xxxx
API_KEY_SPOTIFY=Sp0tifyK3y_123456789
LANG=es_MX.UTF-8
PATH=/usr/local/bin:/usr/bin:/bin:/usr/sbin:/sbin
```

lapply()

Es una función que **nos permite hacer bucles sobre listas**. Tiene la ventaja que no nos exige generar tablas vacías al inicio de un bucle (como lo hacíamos con *for*). Nos devuelve como resultado también una lista.

```
lapply(X, FUN, ...)
```

- **X** : un objeto tipo lista o vector.
- **FUN** : la función que se aplicará a cada elemento.
- **...** : argumentos adicionales que quieras pasarle a la función.

lapply()

```
lapply(X, FUN, ...)
```

- **X**: un objeto tipo lista o vector.
- **FUN**: la función que se aplicará a cada elemento.
- **...**: argumentos adicionales que quieras pasarle a la función.

X: puede ser 1:100 (del uno al 100), puede ser los nombres de los archivos, lo que sea sobre lo que querramos iterar.

FUN: La función que le queremos aplicar a los valores de **X**. Los argumentos adicionales se le pueden aplicar con ...

lapply()

```
lapply(X, FUN, ...)
```

- **X**: un objeto tipo lista o vector.
- **FUN**: la función que se aplicará a cada elemento.
- **...**: argumentos adicionales que quieras pasarle a la función.

Más sobre el FUN:

Al igual que en python, en un lapply podemos aplicar funciones anonimas (lambdas) que se van a ejecutar a todos los elementos de **X**.

lapply()

Ejemplo:

```
palabras <- list("hola", "mundo", "INEGI")  
lapply(palabras, function(x) toupper(x))
```

Función anónima

Salida:

```
[[1]]  
[1] "HOLA"  
  
[[2]]  
[1] "MUNDO"  
  
[[3]]  
[1] "INEGI"
```

lapply()

Ejemplo:

```
palabras <- list("hola", "mundo", "INEGI")  
lapply(palabras, function(x) toupper(x))
```

Función anónima

Salida:

```
[[1]]  
[1] "HOLA"  
  
[[2]]  
[1] "MUNDO"  
  
[[3]]  
[1] "INEGI"
```

lapply()

Si la lista-resultado de un `lapply()` tiene las mismas dimensiones en todos sus elementos, con la función `do.call(rbind, .)` estas listas se pueden convertir en tablas.

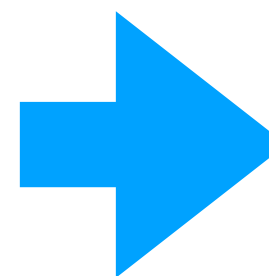
```
library(tibble)

# Tenemos un vector con nombres
personas <- c("Ana", "Luis", "Marta")

# Usamos lapply() para generar una lista de listas
mi_lista <- lapply(seq_along(personas), function(i) {
  tibble(
    nombre = personas[i],
    edad   = 20 + i * 5,
    ciudad = c("CDMX", "Monterrey", "Guadalajara")[i]
  )
})

# Convertimos la lista en tibble con do.call()
mi_tibble <- mi_lista %>% do.call(rbind, .)

mi_tibble
```



nombre	edad	ciudad
<chr>	<dbl>	<chr>
Ana	25	CDMX
Luis	30	Monterrey
Marta	35	Guadalajara

ellmer: cliente unificado

`{ellmer}` es un **cliente unificado para LLMs** hecho por Posit.

- Mismo código para Claude, OpenAI, Gemini, Ollama.
- Maneja autenticación, retry, streaming, tool use.
- Lee las llaves de `.Renviron` automáticamente.
- Soporta **salidas estructuradas** (JSON validado).

```
install.packages("ellmer")  
library(ellmer)
```

chat_google_gemini()

Crear un chat con Gemini es una sola llamada:

```
library(ellmer)

# Asume GOOGLE_API_KEY en .Renviron
chat <- chat_google_gemini(
  model = "gemini-2.0-flash"
)

# Mandar un mensaje y obtener la respuesta
respuesta <- chat$chat("Explicame qué es una API en una oración.")

respuesta
```

El objeto `chat` recuerda el historial por defecto.

Verifica el nombre exacto en `?ellmer` — la API evoluciona rápido.

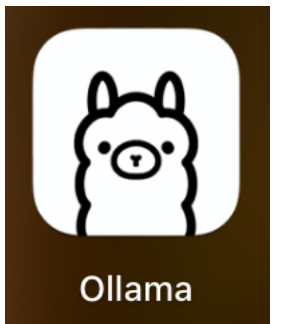
System prompts

Un **system prompt** define el rol y las reglas del modelo antes de empezar.

```
chat <- chat_google_gemini(  
  system_prompt = "  
    Eres un moderador experto en redes sociales.  
    Clasificas mensajes en español como discurso de odio o no.  
    Responde siempre en formato JSON con los campos:  
      - es_discurso_odio (TRUE/FALSE)  
      - razon (string corto)  
  ",  
  model = "gemini-2.0-flash"  
)  
  
chat$chat("Todos los aficionados del otro equipo son unos idiotas.")
```

Buen system prompt = respuestas consistentes y útiles.

Ollama: LLMs locales



Ollama es un servidor que corre LLMs en tu propia máquina.

Ventajas

- **Gratis** (no hay costo por token)
- **Privado** (los datos no salen)
- Sin rate limits
- Funciona sin internet

Desventajas

- Modelos más **pequeños**
- Usa **RAM y GPU**
- Más lento que las APIs de nube
- Calidad menor que GPT-4 o Claude

```
# En terminal, antes de usar desde R:  
ollama pull qwen2.5:7b  
ollama pull llama3.2
```


chat_ollama()

El patrón es idéntico al de Gemini. Solo cambia el constructor:

```
library(ellmer)

chat_local <- chat_ollama(
  system_prompt = "
    Eres un clasificador de discurso de odio y discurso violento.
    Respondes solo TRUE o FALSE si el texto es un texto de odio o violento.
  ",
  model = "qwen3.5:4b"
)

chat_local$chat("Me encantó el partido de ayer.")
# [1] "FALSE"

chat_local$chat("Ojalá se muera ese jugador.")
# [1] "TRUE"
```

Ollama debe estar corriendo (`ollama serve` o la app).

Ejemplo: tuber para YouTube

Flujo típico: autenticación OAuth, luego consulta.

```
library(tuber)
library(tidyverse)

# Autenticación (se abre el navegador la primera vez)
yt_oauth(
  app_id      = Sys.getenv("YT_APP_ID"),
  app_secret  = Sys.getenv("YT_APP_SECRET")
)

# Descargar los comentarios de un video
comentarios <- get_comment_threads(
  filter      = c(video_id = "dQw4w9WgXcQ"),
  max_results = 100
)

comentarios %>%
  as_tibble() %>%
  select(authorDisplayName, textOriginal, likeCount)
```